

Cursor AI Training

Advanced Agentic Engineering Track

Total Duration: 24 Hours — 4 Full Days (6 hours/day)

1. Agenda / Goal

- Provide developers with a comprehensive understanding of Cursor AI as an AI-powered IDE and agentic orchestration platform
- Enable teams to adopt AI-assisted and multi-agent coding practices to improve productivity, code quality, and traceability
- Cover core features: AI chat, inline code generation, codebase-aware editing, refactoring, debugging, and custom rules
- Build reusable agent, prompt-template, and skills frameworks grounded in repository code, requirement documents, and standards
- Establish multi-agent orchestration patterns with automated quality gates, correction loops, and human approval checkpoints
- Apply the full stack to transform a ticket into a validated, independently reviewed, traceable engineering report
- Build hands-on proficiency through five progressively advanced use-case labs, culminating in an end-to-end capstone

2. Prerequisites

- Working knowledge of at least one programming language (JavaScript/Python/Java preferred)
- Familiarity with Git and version control workflows
- Basic understanding of IDEs (VS Code experience helpful, not mandatory)
- Understanding of the team's existing codebase/repo structure and SRS/SDS documentation practices
- Basic familiarity with a ticketing tool (Jira or Azure DevOps) — used in the Day 4 capstone
- Basic familiarity with unit/API testing concepts (PyTest or equivalent helpful, not mandatory)
- No prior AI/LLM or multi-agent orchestration experience required

3. Software / System / License Requirements

- Cursor AI – latest stable version, Enterprise/Team seat with Agent Mode enabled (Windows/Mac/Linux)
- System requirements: Minimum 8GB RAM (16GB recommended), 5GB free disk space
- Git installed and configured; access to a sandbox/training repository (provided by facilitator)
- Node.js / Python (or relevant language runtime) installed, based on project stack
- PyTest (or equivalent test framework) installed for the test-generation and validation labs
- Sample SRS/SDS documents, coding standards, and a historical defect log (sandbox dataset provided)
- MCP server access configured for repository, documentation, and knowledge-graph/vector-store connections (sandbox instance provided — pre-stage before Day 2)
- Read-access API token / sandbox project for Jira or Azure DevOps (provided by facilitator) for the Day 4 capstone
- Stable internet connection (required for AI model and orchestration calls)
- Company SSO/IT approval for Cursor AI and connected MCP/agent tooling (if applicable)

4. Module Overview — 4 Full-Day Schedule

Day 1 — Foundations & Core AI-Assisted Coding

#	Module / Segment	Duration
1	Introduction to Cursor AI	30 mins
2	Cursor Interface & Setup	30 mins
3	AI Chat & Contextual Assistance	60 mins
4	Inline Code Generation & Autocomplete	45 mins
5	Codebase-Aware Editing (Multi-file Context)	60 mins
6	AI-Assisted Refactoring & Debugging	60 mins
7	Custom Rules, Prompts & Team Standards	60 mins
—	Buffer / environment troubleshooting	15 mins

Day total: 360 mins

Day 2 — Reusable Agent, Prompt & Knowledge-Grounding Frameworks

#	Module / Segment	Duration
8	Agent & Skill Architecture Fundamentals	45 mins
9	Use Case Lab 1: Reusable Agents, Prompts & Skills Framework	90 mins
10	Knowledge Grounding Fundamentals	45 mins
11	Use Case Lab 2: Knowledge-Grounded Engineering Agent	90 mins
12	Governance & Security for Agent Frameworks	30 mins
—	Buffer / extended practice	60 mins

Day total: 360 mins

Day 3 — Multi-Agent Orchestration & Quality Gates

#	Module / Segment	Duration
13	Orchestration Patterns Overview	30 mins
14	Use Case Lab 3: Multi-Agent Requirement-to-Test Automation	120 mins
15	Quality Gates & Self-Correction Fundamentals	30 mins
16	Use Case Lab 4: Self-Correcting Agent Orchestration with Quality Gates	120 mins
—	Buffer / multi-agent handoff debugging	60 mins

Day total: 360 mins

Day 4 — End-to-End Engineering Copilot & Program Wrap-Up

#	Module / Segment	Duration
17	Git / Ticketing Integration for Agentic Workflows	30 mins
—	Capstone kickoff & scoping	20 mins
18	Use Case Lab 5 (Capstone): End-to-End Ticket-to-Report Engineering Copilot	150 mins
19	Best Practices, Governance Recap & Enterprise Rollout	30 mins
—	Peer / AI-assisted review of capstone	30 mins
—	Demos & presentations	60 mins
—	Wrap-up, feedback & Q&A	20 mins
—	Buffer / final polish	20 mins

Day total: 360 mins

Total Duration: 24 hours (1,440 mins) across 4 full days

Buffer blocks are sized larger on Days 2–4 to absorb multi-agent debugging and MCP connection setup time — the most failure-prone steps in agentic labs. Day 4 is fully reserved for the capstone build, review, and demos.

5. Modules — Detailed Topics & Subtopics

Day 1 — Foundations & Core AI-Assisted Coding

Module 1: Introduction to Cursor AI

Topics:

- What is Cursor AI and how it differs from traditional IDEs
- Cursor vs. GitHub Copilot vs. other AI coding tools
- Overview of AI-assisted SDLC — from single-prompt coding to multi-agent orchestration

Subtopics:

- Key terminology (LLM, context window, agent, orchestration, MCP)

Hands-on: quick tour and account/license verification.

Module 2: Cursor Interface & Setup

Topics:

- Installing/configuring Cursor AI; connecting the enterprise license/account
- Interface walkthrough (Editor, Sidebar, Command Palette, Chat/Agent Panel)
- Importing existing VS Code settings/extensions; workspace and project setup

Subtopics:

- Connecting MCP servers (repository, docs, ticketing) at the workspace level

Hands-on exercise to follow this module.

Module 3: AI Chat & Contextual Assistance

Topics:

- Using the AI Chat panel for code queries; context modes (file/folder/codebase-level)
- Using @-mentions to reference files, docs, or symbols
- Difference between Chat mode and Composer/Agent mode

Subtopics:

- Effective prompt writing; managing conversation history and context limits

Hands-on exercise and quiz to follow this module.

Module 4: Inline Code Generation & Autocomplete

Topics:

- AI-powered autocomplete (Tab-to-complete); inline generation via natural-language prompts
- Generating functions, classes, and boilerplate code; accepting, rejecting, and editing suggestions

Subtopics:

- Context-aware suggestions based on open files; customizing autocomplete settings

Hands-on exercise to follow this module.

Module 5: Codebase-Aware Editing (Multi-file Context)

Topics:

- Using Composer/Agent mode for multi-file changes; understanding codebase indexing
- Making cross-file edits and feature implementations via AI; reviewing and approving diffs

Subtopics:

- Handling context window limitations in large projects; scoping AI edits safely

Hands-on exercise to follow this module.

Module 6: AI-Assisted Refactoring & Debugging

Topics:

- Refactoring code using natural-language instructions; handling legacy code
- Using AI to identify bugs, explain stack traces, and suggest/apply fixes

- Writing and debugging unit tests with AI assistance

Subtopics:

- Validating AI refactors against existing tests; using terminal output as AI context

Hands-on exercise and quiz to follow this module.

Pacing risk: this module merges the original refactoring and debugging blocks — trim live-coding demos first if time is short.

Module 7: Custom Rules, Prompts & Team Standards

Topics:

- Setting up .cursorrules / project-level AI instructions
- Enforcing coding standards via AI rules; creating reusable prompt templates
- Standardizing AI usage across the development team

Subtopics:

- Team-wide rule configuration and version control

Hands-on exercise to follow this module. This module is the direct foundation for Day 2's reusable agent framework lab.

Day 2 — Reusable Agent, Prompt & Knowledge-Grounding Frameworks

Module 8: Agent & Skill Architecture Fundamentals

Topics:

- Agent vs. rule vs. prompt vs. skill: definitions and when to use each
- Anatomy of a reusable agent (role, inputs, tools, guardrails, outputs)
- Parameterized prompt-template design
- Repository-level rules vs. team-level skills libraries

Subtopics:

- Versioning and reviewing agent/skill definitions like code

Hands-on walkthrough of a sample agent definition.

Module 9 — Use Case Lab 1: Reusable Agents, Prompts & Skills Framework

Lab objective: Create reusable engineering agents, parameterized prompt templates, repository rules, and skills for requirement analysis, test generation, validation, and documentation.

Activities:

- Define agent roles for requirement analysis, test generation, validation, and documentation
- Build parameterized prompt templates with explicit input/output contracts
- Package rules and skills into a shared, version-controlled library
- Test each agent independently against sample inputs

Deliverable: a reusable agent/skill library committed to the shared training repository

Hands-on lab and peer review.

Module 10: Knowledge Grounding Fundamentals

Topics:

- Grounding sources: repository code, SRS/SDS documents, standards, historical defects, knowledge-graph relationships
- Retrieval strategies: semantic search, structured lookups, graph traversal
- Hallucination risk in engineering contexts and why it matters

Subtopics:

- Designing "no unsupported facts" guardrails; citing sources in agent output

Hands-on walkthrough of a grounding configuration.

Module 11 — Use Case Lab 2: Knowledge-Grounded Engineering Agent

Lab objective: Build an agent that uses repository code, SRS/SDS documents, standards, historical defects, and knowledge-graph relationships without inventing unsupported engineering facts.

Activities:

- Connect the agent to repository, document, and knowledge-graph sources via MCP
- Configure retrieval scoping and source citation in agent responses

- Stress-test with out-of-scope questions to verify refusal/grounding behavior

Deliverable: a grounded agent that cites its sources and declines unsupported claims

Hands-on lab and quiz.

Pacing risk: knowledge-graph connection setup is the most failure-prone step in the program — pre-stage connections before the session.

Module 12: Governance & Security for Agent Frameworks

Topics:

- Data privacy and IP handling for connected documents and knowledge graphs
- Enterprise admin controls, audit logs, and access scoping for agent tool use
- Compliance considerations (SOC2, GDPR) as applied to grounded agents

Subtopics:

- Approved vs. restricted knowledge sources; restricted file/folder access policies

Hands-on exercise and quiz to follow this module.

Day 3 — Multi-Agent Orchestration & Quality Gates

Module 13: Orchestration Patterns Overview

Topics:

- Sequential vs. parallel multi-agent pipelines; state and handoff between agents
- Designing agent roles for a pipeline (Validator, Builder, Generator, Reviewer)
- Failure handling and retries in orchestration

Subtopics:

- Choosing orchestration granularity; observability/logging across agent handoffs

Hands-on walkthrough of a sample pipeline diagram.

Module 14 — Use Case Lab 3: Multi-Agent Requirement-to-Test Automation

Lab objective: Orchestrate Requirement Validator, Sequence Builder, Test Generator, API Validator, and Reviewer agents to convert an engineering requirement into validated executable tests.

Activities:

- Configure the Requirement Validator agent to check requirement completeness and testability
- Chain the Sequence Builder agent to derive a test sequence from the validated requirement
- Chain the Test Generator agent to produce executable tests (e.g., PyTest) from the sequence
- Chain the API Validator agent to check generated tests against the target API/interface
- Chain the Reviewer agent to sign off before tests are committed

Deliverable: an executable, validated test suite traced back to its source requirement

Hands-on lab.

Pacing risk: this five-agent chain is the longest build so far — reserve buffer time for handoff debugging.

Module 15: Quality Gates & Self-Correction Fundamentals

Topics:

- Designing automated PASS/FAIL criteria between agent stages
- Correction-loop patterns: who retries, what gets reused, what reruns downstream
- Human-in-the-loop approval checkpoints: where and why to place them

Subtopics:

- Avoiding infinite correction loops; logging gate decisions for traceability

Hands-on walkthrough of a gate configuration.

Module 16 — Use Case Lab 4: Self-Correcting Agent Orchestration with Quality Gates

Lab objective: Implement generator–validator–reviewer orchestration with automated PASS/FAIL gates, correction loops, downstream reruns, and human approval checkpoints.

Activities:

- Add automated gates between the generator, validator, and reviewer agents from Lab 3
- Implement a correction loop that routes FAIL results back to the generator with feedback
- Configure downstream reruns so only affected stages re-execute

- Add a human approval checkpoint before final sign-off

Deliverable: a self-correcting pipeline with gate logs and an approval trail

Hands-on lab and quiz.

Day 4 — End-to-End Engineering Copilot & Program Wrap-Up

Module 17: Git / Ticketing Integration for Agentic Workflows

Topics:

- Connecting Cursor AI/agents to Jira or Azure DevOps via MCP
- Parsing ticket fields into structured agent inputs
- Maintaining traceability from ticket to commit to test to report

Subtopics:

- Git workflow integration (branches, commits, PRs) for agent-generated artifacts

Hands-on exercise to follow this module.

Capstone Kickoff & Scoping

- Assign sandbox tickets to individuals/pairs; review the capstone rubric and traceability requirements

Module 18 — Use Case Lab 5 (Capstone): End-to-End Ticket-to-Report Engineering Copilot

Lab objective: Transform a Jira/Azure DevOps ticket into a requirement bundle, HIL sequence, PyTest, validation report, independent review, checklist, and traceable final engineering report.

Activities:

- Pull a sandbox ticket and extract a structured requirement bundle
- Generate the HIL (human/hardware-in-the-loop) test sequence from the requirement bundle
- Generate PyTest cases and execute them against the sandbox target
- Produce a validation report and route it to an independent reviewer agent
- Compile a checklist and a traceable final engineering report linking ticket → requirement → sequence → tests → results → review

Deliverable: a complete, traceable engineering report package, ready for stakeholder sign-off

Hands-on capstone build.

Pacing risk: this is the full-stack integration of Days 1–3 and the longest single block in the program — buffer is reserved immediately after it.

Module 19: Best Practices, Governance Recap & Enterprise Rollout

Topics:

- Do's and don'ts of multi-agent AI-assisted development; avoiding over-reliance
- Verifying AI/agent output for correctness, security, and traceability
- Rolling out agent/skill libraries and quality gates across a team

Subtopics:

- Common pitfalls in orchestration and how to avoid them

Hands-on discussion.

Peer / AI-Assisted Review, Demos & Wrap-Up

- Peer / AI-assisted review of each capstone deliverable against the rubric
- Demos & presentations of each team's end-to-end engineering report
- Wrap-up, feedback & Q&A